

Analyse des Correspondances Multiples (ACM) : Théorie et Implémentation

DUT : GI-SIR-IDIA / ESTBM USMS

Pr. Abdelaziz QAFFOU

Résumé

Ce document présente l'Analyse des Correspondances Multiples (ACM), une méthode d'analyse factorielle pour l'étude de plusieurs variables qualitatives. Nous expliquerons la théorie sous-jacente, fournirons des exemples d'implémentation en R et Python, et illustrerons la méthode avec un jeu de données concret.

Table des matières

1	Introduction à l'ACM	2
1.1	Domaine d'application	2
2	Théorie mathématique	2
2.1	Étapes principales	2
3	Données d'exemple	3
3.1	Tableau disjonctif complet (extrait)	3
4	Implémentation en R	4
4.1	Code R complet pour l'ACM	4
5	Implémentation en Python	6
5.1	Code Python complet pour l'ACM	6
6	Interprétation des résultats	12
6.1	Analyse des axes	12
6.2	Associations entre modalités	12
6.3	Qualité de la représentation	12
7	Extensions et bonnes pratiques	12
7.1	Pré-traitement des données	12
7.2	Validation des résultats	12
8	Comparaison R/Python	13
9	Conclusion	13

1 Introduction à l'ACM

L'Analyse des Correspondances Multiples (ACM) est une extension de l'Analyse Factorielle des Correspondances (AFC) permettant d'analyser simultanément plusieurs variables qualitatives. Développée dans le cadre de l'analyse des données de Benzécri, elle permet d'étudier les relations entre plusieurs variables qualitatives et de visualiser les associations entre leurs modalités.

1.1 Domaine d'application

- Analyse de questionnaires avec plusieurs questions qualitatives
- Étude des associations entre modalités de plusieurs variables
- Segmentation et typologie d'individus
- Exploration de données catégorielles multivariées

2 Théorie mathématique

Soit un ensemble de Q variables qualitatives observées sur n individus. Chaque variable q a J_q modalités. On construit le **tableau disjonctif complet** (TDC) Z de dimension $n \times J$ où $J = \sum_{q=1}^Q J_q$.

2.1 Étapes principales

1. **Construction du tableau disjonctif complet :**

$$Z = [Z_1 | Z_2 | \dots | Z_Q]$$

où Z_q est la matrice indicatrice de la variable q .

2. **Matrice de Burt :**

$$B = Z^T Z$$

C'est une matrice symétrique bloc contenant les tableaux de contingence croisant toutes les variables deux à deux.

3. **Calcul des profils :**

- Profils lignes : $p_{i|j} = \frac{z_{ij}}{z_{i\cdot}}$
- Poids des lignes : $p_i = \frac{1}{n}$
- Poids des colonnes : $p_j = \frac{z_{\cdot j}}{nQ}$

4. **Matrice des écarts à l'indépendance :**

$$X = D_r^{-1/2} (P - r c^T) D_c^{-1/2}$$

où $P = \frac{1}{nQ} Z$, r est le vecteur des marges lignes, c le vecteur des marges colonnes.

5. **Décomposition en valeurs singulières (SVD) :**

$$X = U \Lambda V^T$$

6. **Coordonnées factorielles :**

- Coordonnées des modalités : $F = D_r^{-1/2} U \Lambda$
- Coordonnées des individus : $\Psi = \frac{1}{Q} Z D_c^{-1} V \Lambda$

7. Inertie :

$$\text{Inertie totale} = \frac{J - Q}{Q}$$

$$\text{Inertie expliquée par l'axe } k = \frac{\lambda_k^2}{\sum \lambda_k^2} \times 100\%$$

3 Données d'exemple

Considérons un jeu de données fictif d'enquête sur les habitudes culturelles de 20 personnes avec 3 variables qualitatives.

TABLE 1 – Jeu de données d'enquête sur les habitudes culturelles

Individu	Âge	Niveau d'études	Activité culturelle préférée
Ind1	18-25	Bac	Cinéma
Ind2	26-35	Licence	Théâtre
Ind3	36-50	Master	Concert
Ind4	51-65	Doctorat	Musée
Ind5	18-25	Bac	Concert
Ind6	26-35	Licence	Cinéma
Ind7	36-50	Master	Musée
Ind8	51-65	Doctorat	Théâtre
Ind9	18-25	Bac	Cinéma
Ind10	26-35	Licence	Concert
Ind11	36-50	Master	Théâtre
Ind12	51-65	Doctorat	Musée
Ind13	18-25	Bac	Concert
Ind14	26-35	Licence	Cinéma
Ind15	36-50	Master	Musée
Ind16	51-65	Doctorat	Théâtre
Ind17	18-25	Bac	Cinéma
Ind18	26-35	Licence	Théâtre
Ind19	36-50	Master	Concert
Ind20	51-65	Doctorat	Musée

3.1 Tableau disjonctif complet (extrait)

TABLE 2 – Extrait du tableau disjonctif complet

Individu	Âge				Niveau d'études				Activité culturelle			
	18-25	26-35	36-50	51-65	Bac	Licence	Master	Doctorat	Cinéma	Théâtre	Concert	Musée
Ind1	1	0	0	0	1	0	0	0	1	0	0	0
Ind2	0	1	0	0	0	1	0	0	0	1	0	0
Ind3	0	0	1	0	0	0	1	0	0	0	1	0
Ind4	0	0	0	1	0	0	0	1	0	0	0	1
Ind5	1	0	0	0	1	0	0	0	0	0	1	0
... (suite des 15 autres individus)												

4 Implémentation en R

4.1 Code R complet pour l'ACM

```
1  # Chargement des packages nécessaires
2  library(FactoMineR)
3  library(factoextra)
4  library(ggplot2)
5  library(ade4)
6  library(reshape2)
7
8  # Création du jeu de données
9  donnees <- data.frame(
10   Individu = paste0("Ind", 1:20),
11   Age = rep(c("18-25", "26-35", "36-50", "51-65"), each = 5),
12   Niveau_etudes = rep(rep(c("Bac", "Licence", "Master", "Doctorat"),
13   each = 5), 1)[1:20],
14   Activite = rep(c("Cinéma", "Théâtre", "Concert", "Musée"), 5)
15 )
16
17 # Afficher les données
18 print("Jeu de données :")
19 print(donnees)
20
21 # ACM avec FactoMineR
22 acm_result <- MCA(donnees[, -1], graph = FALSE)
23
24 # Résumé des résultats
25 print("Résumé de l'ACM :")
26 print(summary(acm_result))
27
28 # Valeurs propres et pourcentage d'inertie
29 valeurs_propres <- acm_result$eig
30 print("Valeurs propres et pourcentage d'inertie :")
31 print(valeurs_propres)
32
33 # Graphique des valeurs propres (Scree plot)
34 fviz_screepLOT(acm_result, addlabels = TRUE,
35   main = "Graphique des valeurs propres (Scree plot)")
36
37 # Graphique des modalités
38 fviz_mca_var(acm_result,
39   col.var = "contrib",
40   gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
41   repel = TRUE,
42   title = "Projection des modalités")
43
44 # Graphique des individus
45 fviz_mca_ind(acm_result,
46   col.ind = "cos2",
47   gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
48   repel = TRUE,
49   title = "Projection des individus")
50
51 # Biplot (individus et modalités)
52 fviz_mca_biplot(acm_result,
53   repel = TRUE,
54   col.var = "#2E9FDF",
```

```

54 col.ind = "#696969",
55 title = "Biplot ACM")
56
57 # Graphique des modalités par variable
58 fviz_mca_var(acm_result,
59 choice = "mca.cor",
60 title = "Corrélations variables-axes")
61
62 # Contribution des modalités aux axes
63 print("Contribution des modalités aux axes :")
64 print(acm_result$var$contrib)
65
66 # Cos2 (qualité de représentation des modalités)
67 print("Cos2 des modalités (qualité de représentation) :")
68 print(acm_result$var$cos2)
69
70 # Cos2 (qualité de représentation des individus)
71 print("Cos2 des individus (qualité de représentation) :")
72 print(head(acm_result$ind$cos2))
73
74 # Coordonnées des modalités
75 print("Coordonnées des modalités :")
76 print(acm_result$var$coord)
77
78 # Coordonnées des individus
79 print("Coordonnées des individus (extrait) :")
80 print(head(acm_result$ind$coord))
81
82 # Description des dimensions
83 print("Description des dimensions par les variables :")
84 dimdesc_result <- dimdesc(acm_result)
85 print(dimdesc_result)
86
87 # Graphique des individus avec coloration par variable
88 fviz_mca_ind(acm_result,
89 habillage = "Age",
90 addEllipses = TRUE,
91 ellipse.type = "convex",
92 palette = "Set1",
93 repel = TRUE,
94 title = "Individus colorés par Âge")
95
96 fviz_mca_ind(acm_result,
97 habillage = "Niveau_etudes",
98 addEllipses = TRUE,
99 ellipse.type = "convex",
100 palette = "Set2",
101 repel = TRUE,
102 title = "Individus colorés par Niveau d'études")
103
104 fviz_mca_ind(acm_result,
105 habillage = "Activite",
106 addEllipses = TRUE,
107 ellipse.type = "convex",
108 palette = "Set3",
109 repel = TRUE,
110 title = "Individus colorés par Activité culturelle")
111

```

```

112 # Nuage des modalités avec étiquettes
113 plot.MCA(acm_result,
114   choix = "ind",
115   invisible = "ind", # Cache les individus
116   title = "Nuage des modalités")
117
118 # Contribution des variables aux axes
119 print("Contribution des variables aux axes :")
120 for(i in 1:ncol(donnees[, -1])) {
121   var_name <- names(donnees[, -1])[i]
122   modalities <- unique(donnees[, -1][, i])
123   cat("\nVariable:", var_name, "\n")
124   cat("Modalités:", paste(modalities, collapse = ", "), "\n")
125 }
126
127 # Tableau de Burt (pour vérification)
128 # Note: Dans MCA, on peut reconstituer la matrice de Burt
129 Z <- acm.dudi(donnees[, -1], scannf = FALSE, nf = 5)
130 print("Dimensions du tableau disjonctif complet :")
131 print(dim(Z$tab))
132
133 # Analyse supplémentaire : V de Cramer entre variables
134 library(vcd)
135 cramer_v <- matrix(NA, ncol = 3, nrow = 3)
136 colnames(cramer_v) <- names(donnees[, -1])
137 rownames(cramer_v) <- names(donnees[, -1])
138
139 for(i in 1:3) {
140   for(j in 1:3) {
141     if(i != j) {
142       tab <- table(donnees[, i+1], donnees[, j+1])
143       cramer_v[i, j] <- assocstats(tab)$cramer
144     }
145   }
146 }
147
148 print("V de Cramer entre les variables :")
149 print(round(cramer_v, 3))
150

```

Listing 1 – ACM en R

5 Implémentation en Python

5.1 Code Python complet pour l'ACM

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from prince import MCA
6 from sklearn.preprocessing import OneHotEncoder
7 import plotly.express as px
8 import plotly.graph_objects as go
9 from plotly.subplots import make_subplots
10

```

```

11 # Création du jeu de données
12 data = {
13     'Individu': [f'Ind{i}' for i in range(1, 21)],
14     'Age': ['18-25', '26-35', '36-50', '51-65'] * 5,
15     'Niveau_etudes': (['Bac']*5 + ['Licence']*5 + ['Master']*5 + ['
16     Doctorat']*5)[:20],
17     'Activite': ['Cinéma', 'Théâtre', 'Concert', 'Musée'] * 5
18 }
19
20 donnees = pd.DataFrame(data)
21 print("Jeu de données :")
22 print(donnees)
23 print()
24
25 # ACM avec le package prince
26 mca = MCA(n_components=2, random_state=42)
27 mca = mca.fit(donnees[['Age', 'Niveau_etudes', 'Activite']])
28
29 # Valeurs propres et inertie
30 print("Valeurs propres (inerties) :")
31 print(mca.eigenvalues_)
32 print()
33
34 print("Pourcentage d'inertie expliquée :")
35 print(mca.percentage_of_inertia_)
36 print()
37
38 print("Inertie expliquée cumulative :")
39 print(mca.cumulative_percentage_of_inertia_)
40 print()
41
42 # Coordonnées des modalités
43 modalites_coords = mca.column_coordinates(donnees[['Age', '
44 Niveau_etudes', 'Activite']])
45 print("Coordonnées des modalités :")
46 print(modalites_coords.round(4))
47 print()
48
49 # Coordonnées des individus
50 individus_coords = mca.row_coordinates(donnees[['Age', '
51 Niveau_etudes', 'Activite']])
52 individus_coords['Individu'] = donnees['Individu']
53 print("Coordonnées des individus (extrait) :")
54 print(individus_coords.head().round(4))
55 print()
56
57 # Contributions
58 print("Contributions des modalités aux axes :")
59 contrib_modalites = mca.column_contributions_.round(4)
60 print(contrib_modalites)
61 print()
62
63 # Graphique des valeurs propres (Scree plot)
64 plt.figure(figsize=(10, 6))
65 inerties = mca.eigenvalues_
66 plt.bar(range(1, len(inerties) + 1), inerties, alpha=0.7, align='
67 center')
68 plt.xlabel('Axe factoriel')

```

```

65 plt.ylabel('Valeur propre (Inertie)')
66 plt.title('Scree plot - Valeurs propres')
67 plt.grid(True, alpha=0.3)
68 plt.show()
69
70 # Biplot ACM avec matplotlib
71 fig, ax = plt.subplots(figsize=(14, 12))
72
73 # Projection des modalités
74 for idx, row in modalites_coords.iterrows():
75     # Déterminer la couleur selon la variable
76     if 'Age' in idx:
77         color = 'blue'
78         marker = 'o'
79     elif 'Niveau_etudes' in idx:
80         color = 'red'
81         marker = 's'
82     else: # Activite
83         color = 'green'
84         marker = '^'
85
86 ax.scatter(row[0], row[1], c=color, marker=marker, s=200, alpha=0.7)
87 ax.annotate(idx, (row[0], row[1]), fontsize=10, color=color,
88             xytext=(5, 5), textcoords='offset points')
89
90 # Projection des individus (en gris)
91 ax.scatter(individus_coords[0], individus_coords[1],
92           c='gray', alpha=0.4, s=50)
93
94 # Ajouter quelques individus avec étiquettes
95 for i in range(0, len(individus_coords), 2): # Un individu sur deux
96     ax.annotate(individus_coords.iloc[i]['Individu'],
97               (individus_coords.iloc[i][0], individus_coords.iloc[i][1]),
98               fontsize=8, color='black', alpha=0.7,
99               xytext=(3, 3), textcoords='offset points')
100
101 # Lignes de référence
102 ax.axhline(y=0, color='k', linestyle='--', alpha=0.3)
103 ax.axvline(x=0, color='k', linestyle='--', alpha=0.3)
104
105 # Configuration du graphique
106 ax.set_xlabel(f'Axe 1 ({mca.percentage_of_inertia_[0]:.1f}%)',
107             fontsize=12)
108 ax.set_ylabel(f'Axe 2 ({mca.percentage_of_inertia_[1]:.1f}%)',
109             fontsize=12)
110 ax.set_title('Biplot ACM - Modalités et individus', fontsize=14)
111 ax.grid(True, alpha=0.3)
112
113 # Légende
114 from matplotlib.lines import Line2D
115 legend_elements = [
116     Line2D([0], [0], marker='o', color='w', markerfacecolor='blue',
117            markersize=10, label='Âge'),
118     Line2D([0], [0], marker='s', color='w', markerfacecolor='red',
119            markersize=10, label='Niveau d\'études'),
120     Line2D([0], [0], marker='^', color='w', markerfacecolor='green',
121            markersize=10, label='Activité'),

```



```

117     Line2D([0], [0], marker='o', color='w', markerfacecolor='gray',
118     markersize=5, label='Individus')
119 ]
120 ax.legend(handles=legend_elements, fontsize=11)
121 plt.tight_layout()
122 plt.show()
123
124 # Graphique séparé des modalités
125 fig, axes = plt.subplots(1, 3, figsize=(18, 6))
126
127 # Âge
128 age_modalites = [m for m in modalites_coords.index if 'Age' in m]
129 for m in age_modalites:
130     axes[0].scatter(modalites_coords.loc[m, 0], modalites_coords.loc[m,
131     1],
132     s=200, alpha=0.7)
133     axes[0].annotate(m.split('_')[1],
134     (modalites_coords.loc[m, 0], modalites_coords.loc[m, 1]),
135     fontsize=11, xytext=(5, 5), textcoords='offset points')
136     axes[0].set_title('Modalités de 1\''Âge', fontsize=12)
137     axes[0].set_xlabel('Axe 1')
138     axes[0].set_ylabel('Axe 2')
139     axes[0].grid(True, alpha=0.3)
140
141 # Niveau d'études
142 niv_modalites = [m for m in modalites_coords.index if 'Niveau_etudes
143 ' in m]
144 for m in niv_modalites:
145     axes[1].scatter(modalites_coords.loc[m, 0], modalites_coords.loc[m,
146     1],
147     s=200, alpha=0.7)
148     axes[1].annotate(m.split('_')[1],
149     (modalites_coords.loc[m, 0], modalites_coords.loc[m, 1]),
150     fontsize=11, xytext=(5, 5), textcoords='offset points')
151     axes[1].set_title('Modalités du Niveau d\''études', fontsize=12)
152     axes[1].set_xlabel('Axe 1')
153     axes[1].grid(True, alpha=0.3)
154
155 # Activité culturelle
156 act_modalites = [m for m in modalites_coords.index if 'Activite' in
157 m]
158 for m in act_modalites:
159     axes[2].scatter(modalites_coords.loc[m, 0], modalites_coords.loc[m,
160     1],
161     s=200, alpha=0.7)
162     axes[2].annotate(m.split('_')[1],
163     (modalites_coords.loc[m, 0], modalites_coords.loc[m, 1]),
164     fontsize=11, xytext=(5, 5), textcoords='offset points')
165     axes[2].set_title('Modalités de 1\''Activité culturelle', fontsize
166     =12)
167     axes[2].set_xlabel('Axe 1')
168     axes[2].grid(True, alpha=0.3)
169
170 plt.tight_layout()
171 plt.show()
172
173 # Graphique interactif avec plotly

```

```

168     fig = make_subplots(rows=1, cols=2,
169     subplot_titles=('Projection des modalités', 'Projection des
individus'),
170     horizontal_spacing=0.15)
171
172     # Modalités
173     for idx, row in modalites_coords.iterrows():
174         if 'Age' in idx:
175             color = 'blue'
176             name = 'Âge'
177         elif 'Niveau_etudes' in idx:
178             color = 'red'
179             name = 'Niveau d\'études'
180         else:
181             color = 'green'
182             name = 'Activité'
183
184     fig.add_trace(
185     go.Scatter(x=[row[0]], y=[row[1]],
186     mode='markers+text',
187     name=name,
188     text=[idx.split('_')[1]],
189     textposition="top center",
190     marker=dict(size=12, color=color),
191     legendgroup=name,
192     showlegend=True if idx == age_modalites[0] or
193     idx == niv_modalites[0] or
194     idx == act_modalites[0] else False),
195     row=1, col=1
196     )
197
198     # Individus
199     fig.add_trace(
200     go.Scatter(x=individus_coords[0], y=individus_coords[1],
201     mode='markers+text',
202     name='Individus',
203     text=individus_coords['Individu'],
204     textposition="top center",
205     marker=dict(size=8, color='gray', opacity=0.6),
206     textfont=dict(size=8)),
207     row=1, col=2
208     )
209
210     # Mise en forme
211     fig.update_xaxes(title_text=f"Axe 1 ({mca.percentage_of_inertia_
[0]:.1f}%)", row=1, col=1)
212     fig.update_yaxes(title_text=f"Axe 2 ({mca.percentage_of_inertia_
[1]:.1f}%)", row=1, col=1)
213     fig.update_xaxes(title_text=f"Axe 1 ({mca.percentage_of_inertia_
[0]:.1f}%)", row=1, col=2)
214     fig.update_yaxes(title_text=f"Axe 2 ({mca.percentage_of_inertia_
[1]:.1f}%)", row=1, col=2)
215
216     fig.update_layout(height=600, width=1200,
217     title_text="ACM - Analyse des Correspondances Multiples",
218     showlegend=True)
219     fig.show()
220

```

```

221 # Analyse de la qualité de représentation (cos2)
222 cos2_individus = mca.row_cosine_similarities_.round(4)
223 print("Cos2 des individus (qualité de représentation) :")
224 print(cos2_individus.head())
225 print()
226
227 cos2_modalites = mca.column_cosine_similarities_.round(4)
228 print("Cos2 des modalités (qualité de représentation) :")
229 print(cos2_modalites.head())
230 print()
231
232 # Individus bien représentés (cos2 > 0.5 sur au moins un axe)
233 individus_bien_repr = cos2_individus[(cos2_individus > 0.5).any(axis
=1)]
234 print("Individus bien représentés (cos2 > 0.5) :")
235 print(individus_bien_repr)
236 print()
237
238 # Tableau disjonctif complet
239 encoder = OneHotEncoder(sparse=False)
240 tdc = encoder.fit_transform(donnees[['Age', 'Niveau_etudes', '
Activite']])
241 tdc_df = pd.DataFrame(tdc, columns=encoder.get_feature_names_out())
242 print("Dimensions du tableau disjonctif complet :", tdc.shape)
243 print()
244
245 # Matrice de Burt
246 burt_matrix = np.dot(tdc.T, tdc)
247 print("Dimensions de la matrice de Burt :", burt_matrix.shape)
248 print()
249
250 # V de Cramer entre variables
251 from scipy.stats import chi2_contingency
252
253 def cramers_v(x, y):
254     confusion_matrix = pd.crosstab(x, y)
255     chi2 = chi2_contingency(confusion_matrix)[0]
256     n = confusion_matrix.sum().sum()
257     phi2 = chi2 / n
258     r, k = confusion_matrix.shape
259     phi2corr = max(0, phi2 - ((k-1)*(r-1))/(n-1))
260     rcorr = r - ((r-1)**2)/(n-1)
261     kcorr = k - ((k-1)**2)/(n-1)
262     return np.sqrt(phi2corr / min((kcorr-1), (rcorr-1)))
263
264 variables = ['Age', 'Niveau_etudes', 'Activite']
265 cramer_matrix = pd.DataFrame(index=variables, columns=variables,
dtype=float)
266
267 for var1 in variables:
268     for var2 in variables:
269         if var1 == var2:
270             cramer_matrix.loc[var1, var2] = 1.0
271         else:
272             cramer_matrix.loc[var1, var2] = cramers_v(donnees[var1], donnees[
var2])
273
274 print("Matrice des V de Cramer :")

```

```
275 print(cramer_matrix.round(3))
276
```

Listing 2 – ACM en Python

6 Interprétation des résultats

Pour notre exemple d'ACM sur les habitudes culturelles, voici comment interpréter les résultats :

6.1 Analyse des axes

- **Axe 1** : Oppose généralement les jeunes avec un niveau Bac (préférant le cinéma) aux personnes plus âgées avec un niveau Doctorat (préférant les musées).
- **Axe 2** : Peut opposer les activités de type "spectacle vivant" (théâtre, concert) aux activités plus "visuelles" (cinéma, musée).

6.2 Associations entre modalités

- La proximité entre "18-25" et "Cinéma" suggère une association entre jeunes et sorties au cinéma.
- La proximité entre "Doctorat" et "Musée" indique une association entre niveau d'études élevé et visites de musées.
- La position de "Concert" au centre peut indiquer une activité appréciée par tous les profils.

6.3 Qualité de la représentation

- Les modalités extrêmes ont généralement une meilleure représentation ($\cos^2$ élevé).
- Les individus dont le profil est "typique" sont mieux représentés.
- L'inertie totale est limitée (caractéristique de l'ACM), mais les premiers axes captent l'essentiel de l'information.

7 Extensions et bonnes pratiques

7.1 Pré-traitement des données

- Regrouper les modalités peu fréquentes
- Éviter les variables avec trop de modalités
- Vérifier l'homogénéité des effectifs par modalité

7.2 Validation des résultats

- Analyser la stabilité des résultats par bootstrap
- Vérifier la cohérence avec d'autres méthodes (clustering, analyse discriminante)
- Interpréter en fonction du contexte métier

8 Comparaison R/Python

TABLE 3 – Comparaison des approches R et Python pour l’ACM

Aspect	R (FactoMineR)	Python (prince)
Installation	<code>install.packages("FactoMineR")</code>	<code>pip install prince</code>
Fonction principale	<code>MCA()</code>	<code>prince.MCA()</code>
Visualisation	Excellente avec <code>factoextra</code>	Bonne avec <code>matplotlib</code> , excellente avec <code>plotly</code>
Sorties détaillées	Très complètes (<code>dimdesc</code> , <code>contributions</code> , <code>cos2</code>)	Complètes avec différentes méthodes
Interactivité	Limitée (sauf avec Shiny)	Excellente avec <code>plotly</code>
Intégration	Bonne avec <code>tidyverse</code>	Excellente avec <code>pandas</code> et <code>scikit-learn</code>
Communauté	Statistique francophone forte	Communauté data science large

9 Conclusion

L’ACM est une méthode puissante pour l’exploration de données catégorielles multivariées. Elle permet de :

- Visualiser simultanément les modalités de plusieurs variables qualitatives
- Identifier les associations entre modalités de variables différentes
- Construire des typologies d’individus
- Réduire la dimensionnalité des données catégorielles

Les implémentations en R et Python offrent des fonctionnalités riches et complémentaires. R avec FactoMineR est particulièrement adapté pour des analyses statistiques approfondies, tandis que Python avec prince s’intègre bien dans des pipelines de data science modernes.

Références

- Benzécri, J.-P. (1973). L’analyse des données. Tome 1 : La taxinomie. Tome 2 : L’analyse des correspondances.
- Lebart, L., Morineau, A., & Warwick, K. M. (1984). Multivariate descriptive statistical analysis.
- Husson, F., Lê, S., & Pagès, J. (2017). Exploratory multivariate analysis by example using R.
- Greenacre, M. (2007). Correspondence analysis in practice.
- Abdi, H., & Valentin, D. (2007). Multiple correspondence analysis.
- Documentation FactoMineR : <https://cran.r-project.org/web/packages/FactoMineR/>
- Documentation prince : <https://github.com/MaxHalford/prince>